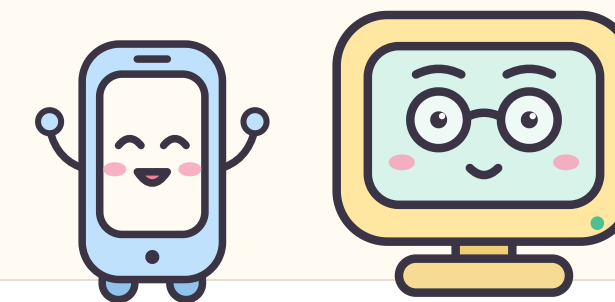
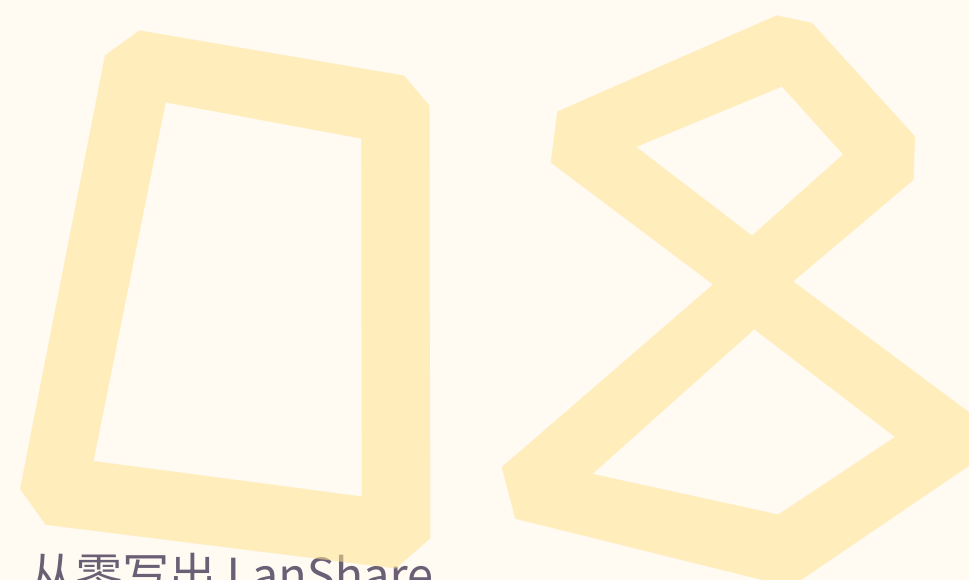




第 8 集 · 约 26 分钟

服务端①：四个基础件

开服务器之前，先回答四个问题



从零写出 LanShare

这一集你会学到

1. 手写命令行解析：while let、闭包、按字符串 match
2. 从一堆网卡地址里，挑出小手机连得上的那一个
3. Mutex：Rust 把锁和它保护的数据装在一起
4. cfg(windows) 加 unsafe：调用 Windows API 取本地时间



先想一个问题

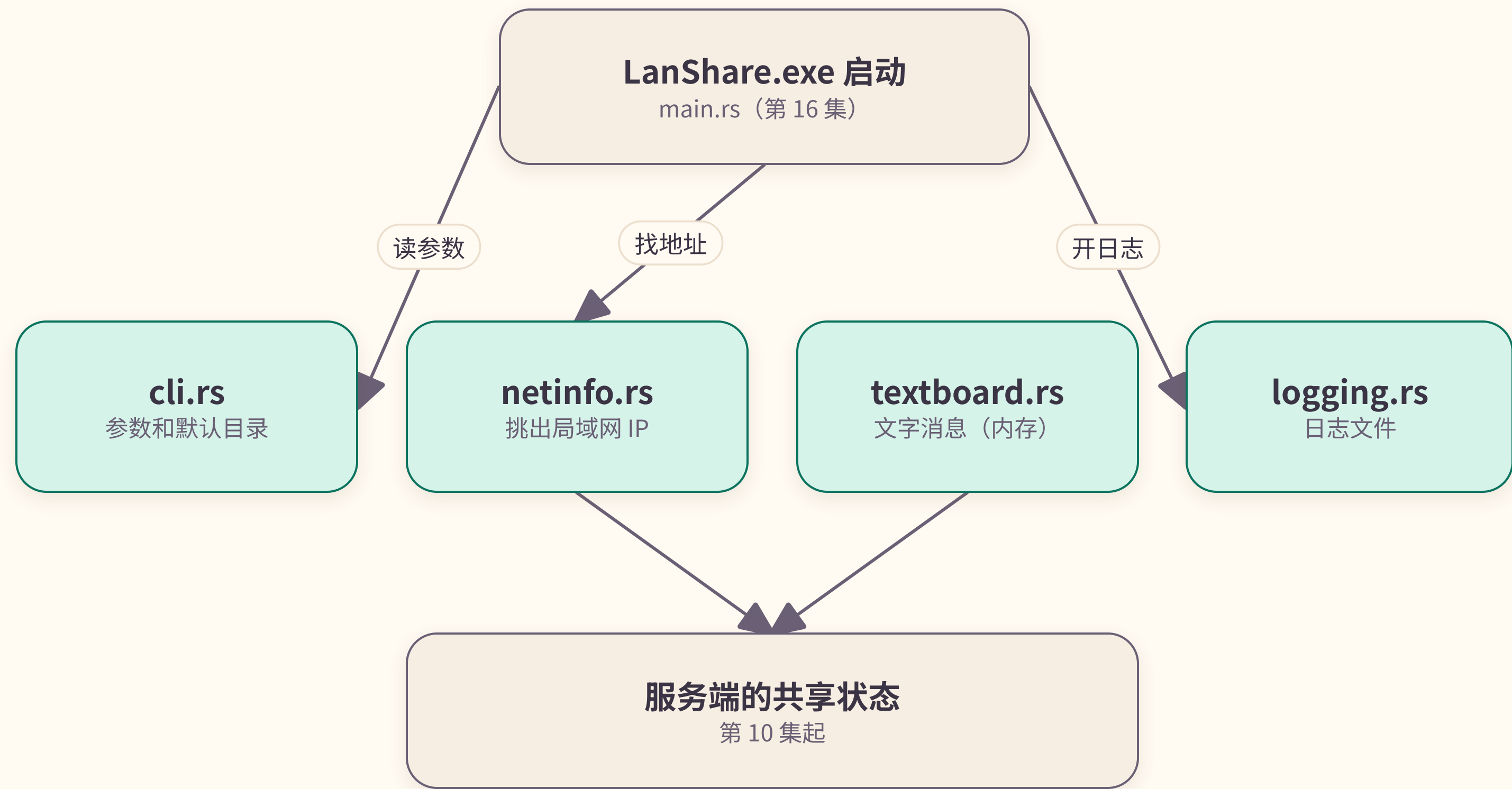
双击 LanShare.exe，到小手机扫码之前，老电脑得先弄清哪几件事？

用户给了什么参数；小手机该连哪个 IP；收到的文字放哪；出了事记在哪。





四个零件，各管一件事





命令行参数：Python 和 Rust

Python v1 • main.py (节选，省略了帮助文字)

```
1 parser = argparse.ArgumentParser(prog="lanshare")
2 parser.add_argument("--port", type=int)
3 parser.add_argument("--dir")
4 parser.add_argument("--no-browser", action="store_true")
5 parser.add_argument("--no-tray", action="store_true")
6 parser.add_argument("--info-file")
7 args = parser.parse_args(argv)
8 args.single_instance = args.port is None and args.dir is None
```

Rust v2 • cli.rs (预告)

```
1 while let Some(arg) = args.next() {
2     match arg.as_str() {
3         "--port" => { /* 再取一个参数，转成 u16 */ }
4         "--dir" => dir = Some(PathBuf::from(value("--dir"?))),
5         "--no-browser" => no_browser = true,
6         "--help" | "-h" | "/"? => return Err(CliError::Help),
7         other => return Err(CliError::Invalid(/* 说明 */)),
8     }
9 }
```



加第一个依赖：dirs

● ● ●

Cargo.toml

crates/lanshare/

第 1-14 行 • 共 14 行

```
1 [package]
2 name = "lanshare"
3 description = "局域网快传：服务端 + Windows 桌面外壳"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [[bin]]
9 name = "LanShare"
10 path = "src/main.rs"
11
12 [dependencies]
13 dirs = "6"
14 lanshare-proto = { path = "../lanshare-proto" }
```



cli.rs: 参数长什么样

cli.rs crates/lanshare/src/ 第 33-52 行 · 共 52 行

```

33     Invalid(String),
34 }
35
36 /// 默认共享文件夹：系统“下载”文件夹下的 LanShare（下载文件夹被挪到别的盘也能找到）。
37 pub fn default_share_dir() -> PathBuf {
38     dirs::download_dir()
39         .or_else(|| dirs::home_dir().map(|h| h.join("Downloads")))
40         .unwrap_or_else(|| PathBuf::from("."))
41         .join("LanShare")
42 }
43
44 /// 程序数据目录（日志、单实例记录）：`%LOCALAPPDATA%\LanShare`。
45 /// 先看环境变量：测试会把它指到临时目录，避免碰本机真实数据。
46 pub fn data_dir() -> PathBuf {
47     std::env::var_os("LOCALAPPDATA")
48         .map(PathBuf::from)
49         .or_else(dirs::data_local_dir)
50         .unwrap_or_else(std::env::temp_dir)
51         .join("LanShare")
52 }

```



cli.rs: 一个循环解析参数

```
cli.rs crates/lanshare/src/ 第 114-133 行 • 共 133 行

114     assert!(matches!(p(&["--port", "70000"]), Err(CliError::Invalid(_))), "超出 u16 范围");
115     assert!(matches!(p(&["--port", "abc"]), Err(CliError::Invalid(_))));
116     assert!(matches!(p(&["--bogus"]), Err(CliError::Invalid(_))));
117 }
118
119 #[test]
120 fn data_dir_follows_localappdata() {
121     // 只读检查: 不改环境变量 (测试是并行跑的)
122     let expected = std::env::var_os("LOCALAPPDATA").map(|v| PathBuf::from(v).join("LanShare"));
123     if let Some(expected) = expected {
124         assert_eq!(data_dir(), expected);
125     }
126     assert!(data_dir().ends_with("LanShare"));
127 }
128
129 #[test]
130 fn default_dir_ends_with_lanshare() {
131     assert!(default_share_dir().ends_with("LanShare"));
132 }
133 }
```



在 lib.rs 里登记

lib.rs crates/lanshare/src/第 1-3 行 · 共 3 行

1

//! 局域网快传 v2：服务端与桌面外壳的库部分（`main.rs` 只负责启动）。

2

3 pub mod cli;





一台电脑，一堆 IPv4 地址

地址	是什么	小手机连得上吗
127.0.0.1	本机回环	不能
169.254.x.x	没从路由器拿到地址时自动分配的	不能
198.18.0.1	代理软件 TUN 模式的虚拟网卡	不能
100.64.x.x	运营商级 NAT、组网软件	一般不能
172.24.16.1	WSL、Hyper-V、Docker 的虚拟网卡	不能
192.168.56.1	虚拟机软件的虚拟网卡	不能
192.168.1.5	真正的 WiFi 网卡	能，就是它





给网段打分：Python 和 Rust

Python v1 • netinfo.py

```
1 _LAN_NETWORKS = [  
2     ipaddress.IPv4Network("192.168.0.0/16"),  
3     ipaddress.IPv4Network("10.0.0.0/8"),  
4     ipaddress.IPv4Network("172.16.0.0/12"),  
5 ]  
6  
7 def _rank(address):  
8     for index, network in enumerate(_LAN_NETWORKS):  
9         if address in network:  
10             return index  
11     return None
```

Rust v2 • netinfo.rs

```
1 fn network_rank(ip: Ipv4Addr) -> Option<u8> {  
2     let [a, b, ..] = ip.octets();  
3     match (a, b) {  
4         (192, 168) => Some(0),  
5         (10, _) => Some(1),  
6         (172, 16..) => Some(2),  
7         _ => None,  
8     }  
9 }
```



加依赖：if-addr



Cargo.toml

crates/lanshare/

第 1-15 行 • 共 15 行

```
1 [package]
2 name = "lanshare"
3 description = "局域网快传：服务端 + Windows 桌面外壳"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [[bin]]
9 name = "LanShare"
10 path = "src/main.rs"
11
12 [dependencies]
13 dirs = "6"
14 if-addr = "0.13"
15 lanshare-proto = { path = "../lanshare-proto" }
```



netinfo.rs: 挑出局域网地址

netinfo.rs

crates/lanshare/src/

第 93-112 行 · 共 115 行

```

93
94     #[test]
95     fn proxy_tun_primary_is_ignored() {
96         let input = ips(&["198.18.0.1", "172.24.16.1", "192.168.1.5"]);
97         assert_eq!(pick_lan_ips(&input, Some("198.18.0.1".parse().unwrap())), ips(&["192.168.1.5", "172.24.16.1"]));
98     }
99
100    #[test]
101    fn virtual_host_adapters_rank_after_real_ones() {
102        let input = ips(&["192.168.56.1", "192.168.137.1", "10.0.0.8", "192.168.40.1", "192.168.1.5"]);
103        assert_eq!(
104            pick_lan_ips(&input, Some("198.18.0.1".parse().unwrap())),
105            ips(&["192.168.1.5", "10.0.0.8", "192.168.56.1", "192.168.137.1", "192.168.40.1"])
106        );
107    }
108
109    #[test]
110    fn primary_missing_from_list_is_added_and_deduped() {
111        assert_eq!(pick_lan_ips(&[], Some("192.168.0.7".parse().unwrap())), ips(&["192.168.0.7"]));
112        let input = ips(&["192.168.1.5", "192.168.1.5"]);
    
```



登记 netinfo

lib.rs

crates/lanshare/src/

第 1-4 行 • 共 4 行

```
1  //! 局域网快传 v2：服务端与桌面外壳的库部分（`main.rs` 只负责启动）。
2
3  pub mod cli;
4  pub mod netinfo;
```





锁和数据：Python 和 Rust

Python v1 • textboard.py (节选)

```
1 class TextBoard:
2     def __init__(self, clock=time.time):
3         self._items = deque(maxlen=MAX_MESSAGES)
4         self._next_id = 1
5         self._lock = threading.Lock()
6
7     def add(self, text):
8         ...
9         with self._lock:
10             item = {"id": self._next_id, ...}
11             self._next_id += 1
12             self._items.append(item)
```

Rust v2 • textboard.rs (预告)

```
1 pub struct TextBoard {
2     inner: Mutex<(u64, VecDeque<TextItem>)>,
3 }
4
5 // add 里面:
6 let mut guard = self.inner.lock().expect("...");
7 let (next_id, items) = &mut *guard;
8 *next_id += 1;
```



加依赖：serde



Cargo.toml

crates/lanshare/

第 1-16 行 • 共 16 行

```
1 [package]
2 name = "lanshare"
3 description = "局域网快传：服务端 + Windows 桌面外壳"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [[bin]]
9 name = "LanShare"
10 path = "src/main.rs"
11
12 [dependencies]
13 dirs = "6"
14 if-addr = "0.13"
15 lanshare-proto = { path = "../lanshare-proto" }
16 serde = { version = "1", features = ["derive"] }
```



textboard.rs: 内存里的消息板

textboard.rs crates/lanshare/src/ 第 56-75 行 · 共 79 行

```
56
57 #[cfg(test)]
58 mod tests {
59     use super::*;
60
61     #[test]
62     fn newest_first_and_bounded() {
63         let board = TextBoard::new();
64         for i in 0..(MAX_MESSAGES + 10) {
65             board.add(&format!("m{i}")).unwrap();
66         }
67         let list = board.list();
68         assert_eq!(list.len(), MAX_MESSAGES);
69         assert_eq!(list[0].text, format!("m{}", MAX_MESSAGES + 9));
70     }
71
72     #[test]
73     fn rejects_blank_and_too_long() {
74         let board = TextBoard::new();
75         assert_eq!(board.add(" \n").unwrap_err(), "empty_text");
```



登记 textboard



lib.rs

crates/lanshare/src/

第 1-5 行 • 共 5 行

```
1  //! 局域网快传 v2：服务端与桌面外壳的库部分（`main.rs` 只负责启动）。
2
3  pub mod cli;
4  pub mod netinfo;
5  pub mod textboard;
```





加依赖：windows-sys 和 tempfile



Cargo.toml

crates/lanshare/

第 1-20 行 • 共 20 行

```
1 [package]
2 name = "lanshare"
3 description = "局域网快传：服务端 + Windows 桌面外壳"
4 version.workspace = true
5 edition.workspace = true
6 publish.workspace = true
7
8 [[bin]]
9 name = "LanShare"
10 path = "src/main.rs"
11
12 [dependencies]
13 dirs = "6"
14 if-addr = "0.13"
15 lanshare-proto = { path = "../lanshare-proto" }
16 serde = { version = "1", features = ["derive"] }
17 windows-sys = { version = "0.61.2", features = ["Win32_System_SystemInformation"] } 临时
18
19 [dev-dependencies]
20 tempfile = "3"
```



logging.rs: 写日志



logging.rs

crates/lanshare/src/

第 74-93 行 · 共 93 行

```
74     assert!(text.contains("收到 a.txt") && text.contains("发送 b.txt"));
75     assert_eq!(text.lines().count(), 2);
76 }
77
78 #[test]
79 #[cfg(windows)]
80 fn local_time_is_human_readable() {
81     let t = local_time();
82     assert_eq!(t.len(), 19, "{t}");
83     assert!(t.starts_with("20") && &t[4..5] == "-" && &t[10..11] == " " && &t[13..14] == ":", "{t}");
84 }
85
86 #[test]
87 fn creates_missing_directory() {
88     let dir = tempfile::tempdir().unwrap();
89     let path = dir.path().join("a").join("b").join("lanshare.log");
90     FileLog::new(path.clone(), false).write("hi");
91     assert!(path.exists());
92 }
93 }
```



登记 logging

lib.rs

crates/lanshare/src/

第 1-6 行 • 共 6 行

```
1  //! 局域网快传 v2：服务端与桌面外壳的库部分（`main.rs` 只负责启动）。
2
3  pub mod cli;
4  pub mod logging;
5  pub mod netinfo;
6  pub mod textboard;
```



四个零件写好了





编译、跑测试

```
Windows PowerShell

PS D:\code\lanshare> cargo build
Updating crates.io index
Locking 29 packages to latest Rust 1.96.0 compatible versions
Adding bitflags v2.13.2
Adding dirs v6.0.0 (available: v7.0.0)
..... (省略 4 行, 都是 Adding)
Adding if-addr v0.13.4 (available: v0.15.0)
..... (省略 7 行, 都是 Adding)
Adding tempfile v3.27.0
..... (省略 14 行, 都是 Adding)
Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 25m 47s

PS D:\code\lanshare> cargo test -p lanshare --lib
Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
Finished `test` profile [unoptimized + debuginfo] target(s) in 6.79s
Running unittests src\lib.rs (target\debug\deps\lanshare-82b7b86216abeef3.exe)
```



选择题

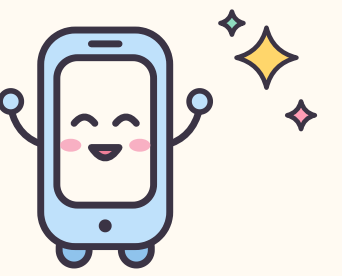
把 `single_instance` 这一行挪到 `dir` 那一行后面，会怎样？

```
1 Ok(Args {  
2     port: port.unwrap_or(DEFAULT_PORT),  
3     dir: dir.unwrap_or(default_dir),  
4     single_instance: port.is_none() && dir.is_none(),  
5     // ...  
6 })
```

A 照常运行，结果一样

B 编译报错：dir 已经被移走了

C 运行时崩溃



选择题

把 `single_instance` 这一行挪到 `dir` 那一行后面，会怎样？

```
1 Ok(Args {  
2     port: port.unwrap_or(DEFAULT_PORT),  
3     dir: dir.unwrap_or(default_dir),  
4     single_instance: port.is_none() && dir.is_none(),  
5     // ...  
6 })
```

A 照常运行，结果一样

结构体字面量按书写顺序求值，`dir` 已经被前一行移走了。

B 编译报错：`dir` 已经被移走了

`unwrap_or` 拿走了 `dir` 的所有权，之后再问 `dir` 是不是空，就是借用一个被移走的值，编译器报 E0382。`port` 是 `Option` 包着 `u16`，也能复制，所以它没问题。

C 运行时崩溃

这类错误 Rust 在编译时就拦下了，根本运行不到。



选择题

文字板的上限是 1 万个字符。为什么判断时用 `text.chars().count()`，而不是 `text.len()`？

A 两个一样，只是写法不同

B `len` 数的是字节，一个汉字占 3 个字节

C `len` 更慢



选择题

文字板的上限是 1 万个字符。为什么判断时用 `text.chars().count()`，而不是 `text.len()`？

A 两个一样，只是写法不同

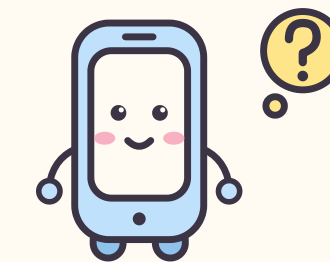
不一样。len 数的是 UTF-8 字节数，chars().count() 数的是字符数。

C len 更慢

恰恰相反，len 直接读长度，chars().count() 要逐个解码。这里看的是对不对，不是快不快。

B len 数的是字节，一个汉字占 3 个字节

用 len 的话，三千多个汉字就超限了。测试里正好 1 万个汉字必须通过，就是为了锁住这一点。



选择题

下面两行，哪一行真的在写文件期间锁住了？

```
1 let _ = self.lock.lock().unwrap_or_else(|e| e.into_inner());  
2 let _guard = self.lock.lock().unwrap_or_else(|e| e.into_inner());
```

A 第一行

B 第二行

C 两行都会



选择题

下面两行，哪一行真的在写文件期间锁住了？

```
1 let _ = self.lock.lock().unwrap_or_else(|e| e.into_inner());  
2 let _guard = self.lock.lock().unwrap_or_else(|e| e.into_inner());
```

A 第一行

单独的下划线不绑定任何变量，守卫当场被丢掉，锁立刻就释放了。

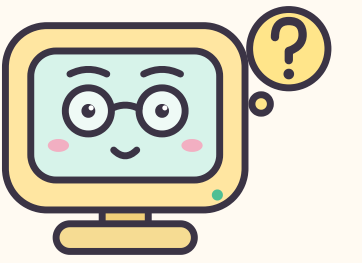
B 第二行

带名字的下划线 guard 是一个真正的变量，守卫一直活到函数结束，锁也一直持有到那时。

C 两行都会

只有带名字的那一行会。这是 Rust 里很有名的坑，后面单实例锁还会遇到。

想一想



netinfo 里，为什么要把 pick_lan_ips 单独写成一个函数，让地址列表从参数传进来，而不是在函数里面直接去查系统？



想一想

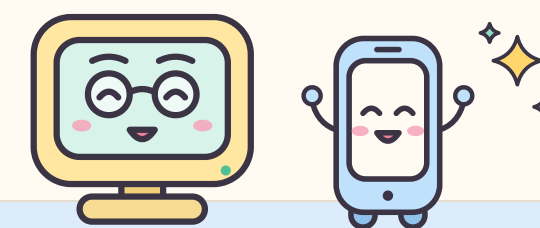
netinfo 里，为什么要把 pick_lan_ips 单独写成一个函数，让地址列表从参数传进来，而不是在函数里面直接去查系统？

为了能测试。查系统的结果取决于这台电脑有几块网卡、开没开代理，写不出确定的断言。把挑选拆成纯函数，就能用构造的地址在任何电脑上得到同样的结果。cli 的 parse 把默认目录当参数传进来，也是同一个思路。



这一集写了什么

- ✓ cli.rs: 一个 while let 循环加一个 match 解析 5 个参数；结构体字面量按书写顺序求值
- ✓ netinfo.rs: 只认三个私有网段，以 1 结尾的往后排，默认路由地址排第一；挑选逻辑写成纯函数好测试
- ✓ textboard.rs: 数据装在 Mutex 里，不拿锁碰不到；按字符数而不是字节数限长
- ✓ logging.rs: cfg(windows) 加 unsafe 取本地时间；日志出错绝不让程序崩
- ✓ let 下划线会立刻丢掉守卫，let 下划线 guard 才会一直持有



下一集

全课完结，感谢收看